

LLaMA 내부 표현 기반 레이어별 Jailbreak 탐지 성능 분석

Layer-wise Jailbreak Detection Performance Analysis Based on Internal Representations of LLaMA

박상은

Sanguen Park

동국대학교 컴퓨터공학과

E-mail: sss1123634@dgu.ac.kr

요약

본 연구는 LLaMA 2 모델의 내부 표현(hidden state)을 활용한 Jailbreak 프롬프트를 탐지 가능성을 분석하였다. 레이어별 성능 비교 결과, 탐지에 유용한 정보는 특정 레이어에 집중되어 나타났으며 영어는 32번 레이어, 한국어는 22번 레이어에서 높은 탐지 성능이 관찰되었다. 또한, Confusion Matrix 분석을 통해 클래스 간 오분류 양상을 분석하였으며, 내부 표현 기반 접근이 Jailbreak 탐지에 활용될 수 있는 가능성을 확인하였다.

1. 서론

1.1 연구 배경

최근 대규모 언어 모델(LLM)은 자연어 이해 및 생성 능력의 비약적인 향상으로 인해 대화형 인공지능, 문서 요약 등 다양한 실생활 서비스에 폭넓게 활용되고 있다. 이에 따라 모델의 안전성과 신뢰성에 대한 중요성 또한 함께 증가하고 있다.

그러나 이러한 발전과 함께, 모델의 안전 정책이나 사용 제한을 우회하려는 Jailbreak 공격이 지속적으로 보고되고 있다. Jailbreak 공격은 유해하거나 비윤리적인 응답을 생성하도록 유도하는 입력 프롬프트로, 모델이 가진 내부 안전 규칙이나 정책을 무력화하여 허위 정보 생성, 차별적·유해한 콘텐츠 출력 등의 문제를 야기할 수 있다.

특히 최근의 Jailbreak 프롬프트는 역할극, 가상 시나리오, 우회적 표현 등 정교한 언어적 전략을 활용하는 방향으로 발전하고 있어, 기존의 키워드 기반 필터링이나 출력 결과 중심의 탐지 방식으로는 효과적인 대응에 한계가 있다. 따라서 입력 단계에서 프롬프트의 위험성을 보다 근본적으로 판별할 수 있는 탐지 방법이 요구된다.

1.2 연구 목적

본 연구의 목적은 LLM의 내부 표현(hidden state)

을 입력 특징으로 활용하여 Jailbreak 프롬프트를 탐지할 수 있는지를 실험적으로 검증하는 것이다. 이를 위해 각 레이어에서 추출한 hidden state를 특징 벡터로 사용하여 분류 실험을 수행하고, 레이어별 탐지 성능 차이를 비교·분석한다.

또한 한국어와 영어 프롬프트를 각각 분석하여 언어별 데이터에서 내부 표현 기반 Jailbreak 탐지 성능의 차이를 비교한다.

이를 통해 본 연구는 hidden state 기반 Jailbreak 탐지의 가능성과 한계를 언어별 관점에서 정리하고, 향후 LLM 보안 연구를 위한 기초적인 실험적 분석 방향을 제시하고자 한다.

2. 연구 방법론

2.1 분석 대상 모델

본 연구는 LLaMA-2-7B-Chat 모델을 분석 대상으로 사용하였다. 해당 모델은 약 70억 개의 파라미터를 갖는 디코더 기반 Transformer 언어 모델로, 총 32개의 레이어로 구성되어 있다.

각 레이어는 입력 프롬프트에 대한 hidden state를 단계적으로 갱신하며, 의미 정보를 축적한다. 본 연구에서는 레이어별 hidden state가 Jailbreak 탐지 성능에 미치는 영향을 비교하기 위해, 모든 레이어에서 hidden state를 개별적으로 추출하여 분석에

활용하였다.

2.2 데이터셋 구성 및 전처리

본 연구는 jailbreak 탐지 실험을 위해 WildJailbreak 데이터셋을 사용하였다. 해당 데이터셋은 정상 프롬프트와 다양한 형태의 jailbreak 프롬프트를 포함하고 있어 관련 연구에서 널리 활용되고 있다.

데이터의 균형을 고려하여 4-class 분류 체계를 구성하였으며, 각 클래스당 100개씩 총 400개의 프롬프트를 사용하였다. 클래스는 Vanilla Benign, Vanilla Harmful, Adversarial Benign, Adversarial Harmful의 네 가지 유형[1]으로 구성된다.

또한, 언어적 차이에 따른 내부 표현 차이를 분석하기 위해 한국어 데이터셋을 추가로 구성하였다. 한국어 프롬프트는 기존 영어 프롬프트를 GPT-5.1을 활용해 번역한 후, 직역으로 인한 부자연스러운 표현이나 불필요한 요소를 제거하는 방식으로 직접 검수하였다. 이를 통해 영어 프롬프트와 의미적으로 대응되는 한국어 데이터셋을 구성하였다.

2.3 특징 벡터 구성 및 분류 방법

본 연구에서는 LLaMA-2-7B-Chat의 각 레이어에서 추출한 hidden state를 분류기의 입력 특징으로 사용하였다. 고정 길이 벡터 구성을 위해 last-token과 mean-pooling 방식을 적용하고, 두 방식에 따른 레이어별 jailbreak 탐지 성능 차이를 비교하였다.

분류기로는 Logistic Regression을 사용하였다. 이는 분류기 복잡도로 인한 성능 향상을 최소화하여, hidden state 자체의 분별력을 레이어별로 비교하기 위함이다. 모든 레이어와 특징 구성 방식에 동일한 분류기를 적용하여, 성능 차이가 hidden state 표현과 레이어 특성에서 기인하도록 실험을 설계하였다.

3. 실험 설정

3.1 데이터 분할 및 실험 환경

전체 데이터셋은 학습 데이터와 테스트 데이터로 각각 80%와 20%의 비율로 분할하였으며, 클래스 불균형을 방지하기 위해 stratified split을 적용하였다. 레이어별 성능 비교의 일관성을 확보하기 위해 모든 실험에서 동일한 데이터 분할을 사용하였다.

실험은 Google Colab 환경에서 수행하였으며, NVIDIA A100 GPU를 사용하였다. 토큰 길이에 따른

영향 최소화를 위해 사전 분석을 통해 언어별 최대 토큰 길이와 배치 크기를 설정하였다. 영어 프롬프트는 max length 512, batch size 16, 한국어 프롬프트는 max length 1024, batch size 8로 설정하였다.

3.2 평가 지표

레이어별 jailbreak 탐지 성능 평가는 Accuracy, Macro F1-score, 그리고 Normalized Confusion Matrix를 기준으로 수행하였다.

Accuracy와 Macro F1-score는 전반적인 분류 성과와 클래스 균형을 고려한 성능 비교에 사용되었으며, Confusion Matrix는 레이어별로 어떤 클래스 간 오분류가 발생하는지를 정성적으로 분석하는 데 활용하였다.

4. 실험 결과

4.1 Pooling 방식에 따른 분류 성능 비교

hidden state의 특징 구성 방식에 따른 Jailbreak 탐지 성능을 비교한 결과, 영어와 한국어 데이터셋 모두에서 mean-pooling 방식이 last-token 방식보다 전반적으로 높은 Accuracy와 F1-score를 보였다. 이러한 경향은 대부분의 레이어 구간에서 일관되게 관찰되었으며, 특히 중·후반 레이어에서 성능 차이가 더욱 뚜렷하게 나타났다.

이는 last-token 방식이 프롬프트의 마지막 토큰 정보만을 반영하는 반면, mean-pooling 방식은 전체 토큰의 정보를 평균화하여 문장 전체의 의미를 보다 안정적으로 반영하기 때문으로 해석된다.

이러한 결과를 바탕으로, 이후 레이어별 성능 분석 및 혼동 행렬 기반 분석에서는 pooling 방식에 따른 영향을 최소화하기 위해 mean-pooling 방식을 기본 설정으로 사용하였다.

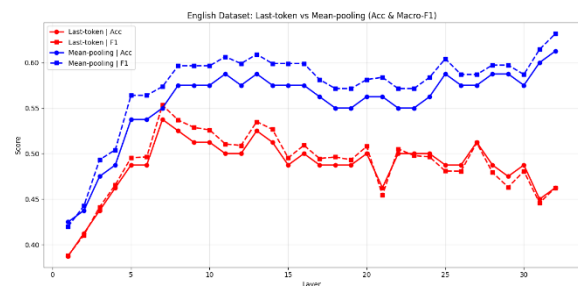


그림 1. 영어 데이터셋에서의 레이어별 분류 성능 변화 (Last-token vs Mean-pooling)

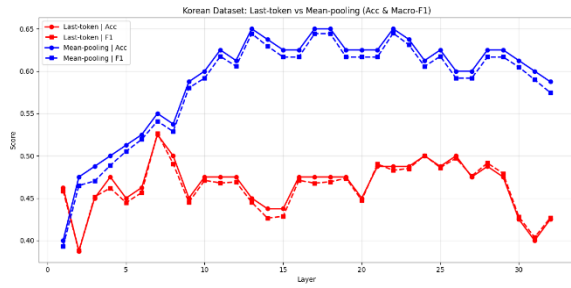


그림 2. 한국어 데이터셋에서의 레이어별 분류 성능 변화 (Last-token vs Mean-pooling)

4.2 레이어별 Jailbreak 탐지 성능 분석

mean_pooling 방식을 적용한 레이어별 분류 성능을 분석한 결과, 영어와 한국어 데이터셋 모두에서 레이어에 따라 Accuracy 및 F1-score가 유의미하게 변화하는 양상이 관찰되었다.

영어 데이터셋에서는 11, 13, 25, 32번 레이어에서 비교적 높은 성능이 관찰되었고, 한국어 데이터셋에서는 13, 17, 18, 22번 레이어에서 상대적으로 높은 분류 성능을 보였다. 이러한 결과는 Jailbreak 탐지에 유용한 정보가 모든 레이어에 균등하게 분포하지 않으며, 특정 레이어의 내부 표현이 분류에 더 효과적으로 활용될 수 있음을 시사한다.

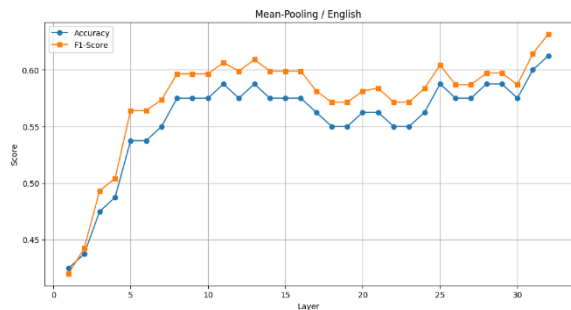


그림 3. 영어 데이터셋의 레이어별 분류 성능 변화 (Mean-pooling)

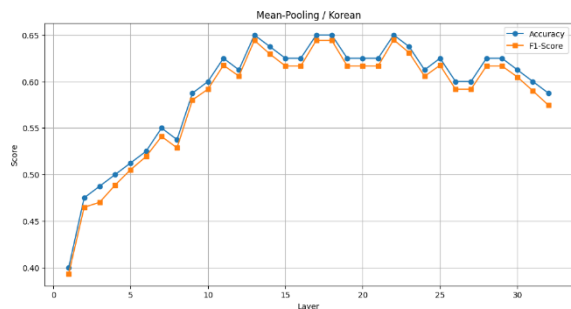


그림 4. 한국어 데이터셋의 레이어별 분류 성능 변화 (Mean-pooling)

4.3 Confusion Matrix 기반 오분류 유형 분석

본 절에서는 4.2절에서 상대적으로 높은 성능을 보인 레이어들을 대상으로 confusion matrix를 비교하여 레이어별 분류 특성을 분석하였다. 그 결과 영어 데이터셋에서는 32번 레이어, 한국어 데이터셋에서는 22번 레이어가 가장 우수한 성능을 보였으며, 본절에서는 이 두 레이어를 중심으로 오분류 유형을 분석하였다.

특히 한국어 데이터셋의 경우, 22번 레이어에서 Jailbreak 프롬프트에 해당하는 adversarial_harmful 클래스에 대해 0.75의 탐지 비율이 관찰되었다. 이는 내부 표현(hidden state) 기반 방법이 실질적인 Jailbreak 탐지에도 일정 수준의 효과를 가질 수 있음을 시사한다.

본 연구의 4-class는 유해성(harmful/benign)과 우회성(adversarial/vanilla)이 결합된 구조이므로, 단순 정확도보다 클래스 간 전이 양상을 분석하는 것이 중요하다. 이에 따라 주요 오분류 유형을 ①~⑦로 구분하여, 각 전이에서 관찰된 언어적·구조적 원인을 분석하였다.

①vanilla_benign → vanilla_harmful

본 오류는 한국어 데이터셋에서만 관찰되었으며, benign 의도의 프롬프트가 유해 요청으로 오분류된 사례이다. 이는 유해 키워드와 부정 표현이 결합된 완곡한 서술 방식에서 안전 의도가 충분히 반영되지 않아, 모델이 유해 신호를 과대 인식한 결과로 해석된다.

②vanilla_benign → adversarial_benign

본 오류는 영어와 한국어 데이터셋 모두에서 관찰되었으며, benign한 의도임에도 게임·학술·비유적 맥락에서 위험 단어가 포함될 경우 우회적 유해 요청으로 오인되는 경향을 보였다. 이는 문맥적 안전성보다 구조적 공격 신호가 우선적으로 반영된 결과로 해석된다.

③vanilla_harmful → vanilla_benign

본 오류는 주로, 한국어 데이터셋에서 관찰되었으며, 유해 요청이 정보 서술 또는 요청의 형태로 완곡하게 표현될 때 benign으로 오분류되는 사례이다. 이로 인해 모델이 유해성을 충분히 반영하지 못한 것으로 해석된다.

④vanilla_harmful → adversarial_benign

본 오류는 영어와 한국어 데이터셋 모두에서 관찰되었으며, 짧고 명령형 구조의 입력에서 우회성과

유해성이 동시에 정확히 반영되지 않아 유해 요청이 adversarial_benign으로 오분류되는 사례이다. 이는 hidden state 기반 방식의 한계 또는 관련 학습 데이터 부족에서 기인한 것으로 판단된다.

⑤adversarial_benign → adversarial_harmful

본 오류는 영어와 한국어 데이터셋 모두에서 관찰되었으며, 우회적이지만 benign한 프롬프트가 유해한 공격으로 오분류된 사례이다. 이는 과도하게 통제적인 구조가 포함될 경우, 구조적 공격 신호가 우선 인식되는 경향에 따른 결과로 해석된다.

⑥adversarial_harmful → vanilla_benign

본 오류는 한국어 데이터셋에서만 관찰되었으며, 허구 세계관 기반 설정이 서사적 배경 설명으로 인식되면서 우회성과 유해성이 충분히 반영되지 못한 사례이다.

⑦adversarial_harmful → adversarial_benign

본 오류는 영어와 한국어 데이터셋 모두에서 관찰되었으며, 허구적 세계관이나 서사적 설정으로 인해 유해성 신호가 약화되면서 adversarial_benign으로 오분류된 경우이다.

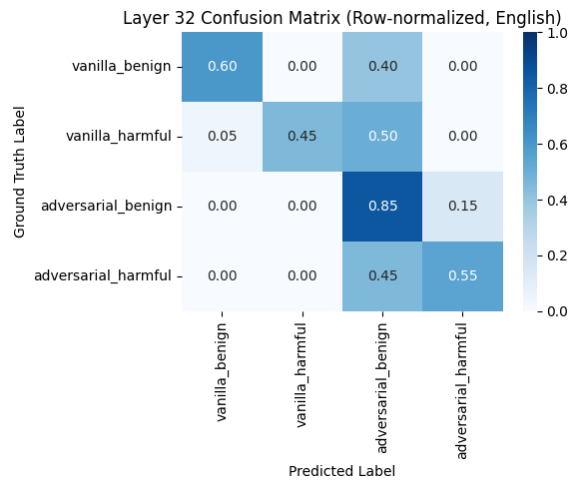


그림 5. 영어 데이터셋에서 최적 레이어(Layer 32)의 Confusion Matrix (Row-normalized)

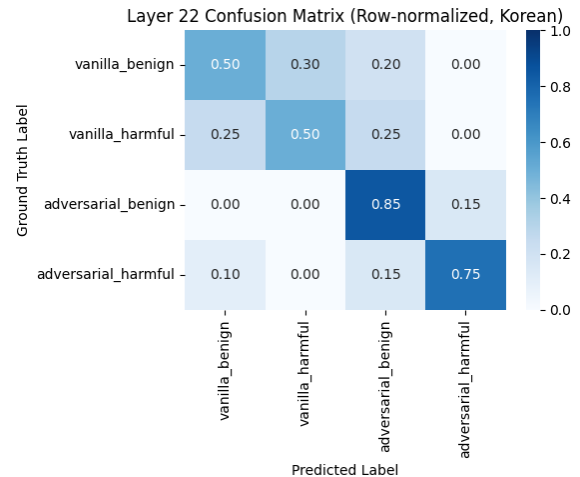


그림 6. 한국어 데이터셋에서 최적 레이어(Layer 22)의 Confusion Matrix (Row-normalized)

5. 결론

본 연구에서는 LLaMA-2-7B-Chat 모델의 내부 표현(hidden state)을 활용하여 Jailbreak 프롬프트 탐지가 가능한지를 분석하였다. 레이어별 hidden state를 기반으로 분류 성능을 비교한 결과, Jailbreak 탐지에 유용한 정보는 모든 레이어에 균등하게 분포하지 않으며, 특정 레이어의 내부 표현이 분류에 더 효과적으로 활용될 수 있음을 확인할 수 있었다. 또한, Confusion Matrix 분석을 통해 클래스 간 오분류 양상을 구체적으로 확인하였고, 한국어 데이터셋의 경우 22번 레이어에서 adversarial_harmful 클래스에 대해 0.75의 탐지 비율이 관찰되어 내부 표현 기반 접근의 실효성을 확인할 수 있었다.

본 연구는 단일 모델과 제한된 규모의 데이터셋을 기반으로 수행되었음에도, 특정 레이어에서 의미 있는 Jailbreak 탐지 성능이 관찰되었다는 점에서 의미가 있다. 나아가 Confusion Matrix를 통해 확인된 오분류 양상을 반영하여 학습 데이터를 보강하거나 분류 전략을 개선할 경우, Jailbreak 탐지 성능을 더욱 향상시킬 수 있을 것으로 기대된다.

참 고 문 헌

[1] Jiang, L., Rao, K., Han, S., et al. "WildTeaming at Scale: From In-the-Wild Jailbreaks to (Adversarially) Safer Language Models." arXiv preprint arXiv:2406.18510 (2024).